

★ WEEK 7 · PROBLEM SET

Finance Applications: Three Hands-On Problems

Gini by Hand · Forest Vote · Boosting Steps

Each problem: Statement → Principles & Lab Guide → Hand Calculation → Claude Code Prompt

Small numbers you can compute with a pencil — then a metaprompt that lets Claude Code scale the same idea to real data. Hand first, machine second.

Every Problem Follows the Same Four Steps

Work top to bottom — do not skip the hand calculation, it is where the understanding lives

1 · Statement

the question

A concrete finance scenario with a tiny dataset you can see at a glance.

one slide

2 · Principles

the tools

The formulas involved and a short lab guide: what to compute, in what order.

one slide

3 · Hand Calc

pencil first

Work every number by hand in a table.
Small n , clean arithmetic, no calculator fear.

1–2 slides

4 · Claude Code

scale it up

A metaprompt: paste it into Claude Code and it builds the full lab on real data.

one slide

Hand first, machine second — you check the machine, never the other way around.

Where the Data Comes From

Public / in-script sources first — the prompts generate or fetch free data; CSVs are hand-calc references

Problem	Primary source (public · no signup)	Provided fallback file
P1 · Gini split	§ 5 in-script set (make_classification, fixed seed)	fml_w7_p1_loans.csv (hand-calc)
P2 · Forest vote	§ 5 in-script set — same data, bagging demo	fml_w7_p2_samples.csv (hand-calc)
P3 · Boosting	§ 5 in-script set + OpenML 'creditcard' (optional)	— numbers printed on the slides
Real-data extension	sklearn fetch_openml('creditcard') — free, no key	—

Course rule: if data is free online (yfinance, FRED, OpenML) or generated in-script, the prompt uses it.

P1

PROBLEM 1 · GINI

Which Question Is Worth Asking?

Ten loans, two candidate splits — let the purity meter decide

Ten Loans, Two Possible Questions

6 repaid, 4 defaulted. Candidate splits: 'income > \$50k?' vs 'has a credit card?'

	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10
income > 50k?	Y	Y	Y	Y	Y	N	N	N	N	N
has card?	Y	N	Y	N	Y	N	Y	Y	N	N
outcome	R	R	R	R	R	R	D	D	D	D

Notation

- R = repaid, D = defaulted.
- Root mix before any question: 6R / 4D.

Your deliverables

- Gini of the root and of every child group.
- Weighted Gini after each question; the gains.
- The split a tree would choose — and why.

The Tool: A Purity Meter for Groups

Gini 0 = all one class; 0.5 = a coin flip — a question's value is the purity it buys

$$G = 1 - \sum_k p_k^2 \quad G = 0: \text{ perfectly pure} \quad G = 0.5: \text{ a 50/50 coin flip}$$

Reading the meter

- Compute p for each class in a group, square, sum, subtract from 1.
- A pure leaf (5R/0D) scores exactly 0 — nothing left to ask.
- The question's value = root Gini – weighted child Gini (the 'gain').

Lab guide — per question

- Step 1: split the 10 rows into the question's two groups.
- Step 2: Gini per group, then weight by group size.
- Step 3: compare gains across questions; biggest gain wins.

Two Questions Through the Meter

Income buys 0.32 of purity; the card question buys exactly nothing

Group	Mix (R/D)	Gini = $1 - \sum p^2$	Weighted
Root	6 / 4	$1 - (0.36 + 0.16) = 0.48$	—
income YES	5 / 0	$1 - (1.00 + 0) = 0.00$	0.5×0.00
income NO	1 / 4	$1 - (0.04 + 0.64) = 0.32$	$0.5 \times 0.32 \rightarrow \text{total } 0.16$
card YES	3 / 2	$1 - (0.36 + 0.16) = 0.48$	0.5×0.48
card NO	3 / 2	$1 - (0.36 + 0.16) = 0.48$	$0.5 \times 0.48 \rightarrow \text{total } 0.48$

root 0.48 $\xrightarrow{\text{income split}}$ 0.16 (gain 0.32) card split: 0.48 (gain 0)

The Tree Asks About Income — and Ignores the Card

Gain 0.32 vs gain 0: a question that leaves both children as mixed as the parent is worthless

0.48

Root impurity — a 6/4 mix, close to a coin flip

0.16

After the income question — one child perfectly pure

0.32

The income question's gain — the biggest on offer

0.00

The card question's gain — both children still 3/2

What the tree does next

- The pure YES-leaf stops; the 1R/4D NO-branch splits again.
- Greedy, level by level — exactly your hand procedure.

The finance reading

- 'Has a card' correlates with nothing here — a decoy clue.
- Gini is the tree's built-in defense against decoys.

Scale It: Watch a Real Tree Choose Its Splits

Paste the metaprompt — Claude Code grows the tree and prints every Gini decision

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) generates the class dataset (make_classification,  
   20000 x 20, weights=[0.99, 0.01], random_state=42);  
2) fits DecisionTreeClassifier(max_depth=3) on a  
   train split;  
3) prints the tree with export_text and, for the root,  
   reproduces the Gini arithmetic by hand in numpy:  
   parent Gini, both children, weighted average, gain;  
4) sweeps max_depth 1..12 and plots train vs validation  
   F1 (the U-curve from class);  
5) three-sentence verdict: which feature won the root,  
   what gain it bought, and where the U-curve says to  
   stop asking questions.
```

[OUTPUT] What to expect

```
root split on one of the 6  
informative features  
numpy Gini = sklearn's  
reported impurities  
validation F1 peaks at a  
shallow-ish depth
```

```
Your 10-loan meter, run  
on 20,000 rows.
```

P2

PROBLEM 2 · FOREST VOTE

Three Histories, One Ballot

Bootstrap the same ten loans — and watch the trees disagree productively

Three Trees, Three Bootstrap Samples

Each tree trained on 6 rows drawn WITH replacement from P1's ten loans. Query: income $\leq$ 50k, card = Y

Tree	Bootstrap sample	Rule it learned
T1	A1 A2 A4 A7 A9 A9	income > 50k $\rightarrow$ Repaid; else $\rightarrow$ Default
T2	A3 A5 A6 A6 A8 A10	income > 50k $\rightarrow$ Repaid; else: card = Y $\rightarrow$ Default, card = N $\rightarrow$ Repaid
T3	A1 A2 A6 A6 A6 A9	income > 50k $\rightarrow$ Repaid; else $\rightarrow$ Repaid (its low-income rows were 3R/1D)

Your deliverables

- A6's appearances per sample · each tree's verdict on the query · the majority vote.

The Tool: Resample, Train, Vote

Bagging gives every tree its own remembered history — diversity by design

$$\hat{y} = \text{majority}\{T_1(q), T_2(q), T_3(q)\} \quad \text{each } T_m \text{ trained on its own bootstrap sample}$$

Why the trees differ

- Drawing WITH replacement duplicates some rows, drops others.
- A6 is the swing voter: seen 0, 2, or 3 times across samples.
- Different histories → different low-income leaves → healthy disagreement.

Lab guide — three steps

- Step 1: count A6's appearances per sample.
- Step 2: push the query through each tree's rule.
- Step 3: tally the ballots; majority wins.

The Ballot, Row by Row

Query ($\text{income} \leq 50\text{k}$, $\text{card} = Y$) — two trees say Default, one says Repaid

Tree	A6 appears	Walking the query through the rule	Verdict
T1	0×	$\text{income} \leq 50\text{k} \rightarrow$ its low-income rows were all defaulters	Default
T2	2×	$\text{income} \leq 50\text{k}$, $\text{card} = Y \rightarrow$ the A8 path (a defaulter)	Default
T3	3×	$\text{income} \leq 50\text{k} \rightarrow$ a 3R/1D leaf, dominated by A6 copies	Repaid

T_1 : Default T_2 : Default T_3 : Repaid $\implies$ vote 2-1: DEFAULT

Vote 2–1: DEFAULT — and the Dissent Was the Point

T3 over-trusted its triple dose of A6; the committee outvoted the fluke

0 · 2 · 3

A6's appearances —
one resampling, three
worldviews

2 – 1

The ballot: Default
wins, dissent noted

1 tree?

Trained on sample 3
alone you'd approve
the loan

200+

Real forests: enough
voters that no fluke
row rules

The bagging lesson

- Each tree overfits its own history a little — differently.
- Averaging different overfittings $\approx$ cancelling the noise.

Week 1, again

- Variance reduction without adding bias — the free lunch of diversity.
- Feature subsampling (randomness #2) pushes the same idea further.

Scale It: From 3 Ballots to 200

Paste the metaprompt — Claude Code raises the full forest and measures the crowd effect

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) generates the class dataset (make_classification,  
   20000 x 20, weights=[0.99, 0.01], random_state=42),  
   split train/validation/test;  
2) fits RandomForestClassifier for n_estimators in  
   [1, 5, 25, 100, 200] and plots validation F1 vs  
   the number of trees;  
3) measures tree disagreement: for 200 trees, the  
   average pairwise prediction disagreement rate on  
   validation rows;  
4) reports the out-of-bag score (oob_score=True) and  
   explains in one line what OOB reuses;  
5) three-sentence verdict: how F1 and disagreement  
   evolve as trees are added, in ballot language.
```

[OUTPUT] What to expect

```
F1 climbs steeply then  
plateaus by ~100 trees  
healthy disagreement stays  
nonzero - that is the fuel  
OOB score ~ test score,  
free of charge
```

```
Your 2-1 ballot, held with  
200 voters.
```

P3

PROBLEM 3 · BOOSTING

Melt the Error, Half a Step at a Time

Four loans, two rounds, $\eta = 0.5$ — geometric convergence by hand

Four Loans, a Mean, and a Careful Step

Target y = default indicator (0, 0, 1, 1). Start from the mean; boost twice with $\eta = 0.5$

Loan	L1	L2	L3	L4
income > 50k?	Y	Y	N	N
y (defaulted?)	0	0	1	1

The weak learner

- Each h is a one-question stump on income.
- It fits the CURRENT residuals perfectly per group.

Your deliverables

- F_0 (the mean) and its residuals; then F_1 and F_2 with $\eta = 0.5$.
- The SSE after each round.
- One sentence: why not take the full step ($\eta = 1$) and finish in one round?

The Tool: Add a Fraction of the Correction

Each round: read the leftover error, fit a stump to it, add half of it

$$F_m = F_{m-1} + \eta h_m \quad \eta = 0.5, \quad h_m \text{ trained on the leftover errors}$$

Why the error melts geometrically

- The stump fits residuals exactly, but we add only η of it.
- So $(1 - \eta)$ of each residual survives: errors shrink by $(1-\eta)^2 = 25\%$ per round in SSE.
- Small η = slow, safe melt; that is the whole secret.

Lab guide — per round

- Step 1: residuals $r = y - F$.
- Step 2: stump h = group-average of r (here: r itself).
- Step 3: $F \leftarrow F + 0.5 \cdot h$; recompute SSE. Repeat once more.

Two Rounds, Every Number

From knowing nothing (0.5 everywhere) to nearly the truth in two half-steps

Step	F (L1, L2, L3, L4)	Residuals $y - F$	SSE
$F_0 = \text{mean}(y) = 0.5$	(0.50, 0.50, 0.50, 0.50)	(-0.5, -0.5, +0.5, +0.5)	1.00
$h_1 = \text{residual stump}; F_1 = F_0 + 0.5 \cdot h_1$	(0.25, 0.25, 0.75, 0.75)	(-0.25, -0.25, +0.25, +0.25)	0.25
$h_2 = \text{new stump}; F_2 = F_1 + 0.5 \cdot h_2$	(0.125, 0.125, 0.875, 0.875)	(± 0.125)	0.0625

SSE: 1.00 $\xrightarrow{\text{round 1}}$ 0.25 $\xrightarrow{\text{round 2}}$ 0.0625 each round keeps $(1 - \eta)^2 = 25\%$

SSE 1.00 → 0.25 → 0.0625

Each careful half-step keeps exactly 25% of the error — and that caution is the feature

× 0.25

SSE multiplier per round at $\eta = 0.5$ — clockwork

$\eta = 1?$

One perfect step HERE — and pure noise-chasing on real data

0.875

F_2 for the defaulters — creeping toward the truth of 1

2 knobs

η and the number of rounds — watched on validation

Why $\eta = 1$ is a trap

- Our stump was perfect because the toy has no noise.
- Real residuals are half noise — full steps copy the noise in.

Forest vs. boosting, felt

- P2's forest averaged independent mistakes (variance).
- P3's relay attacks the remaining miss itself (bias).

Scale It: The Melt Curve on Real Data

Paste the metaprompt — Claude Code traces staged predictions and the $\eta \times$ rounds trade

[INPUT] Prompt · Claude Code

Build a single Colab-ready Python script that:

- 1) generates the class dataset (`make_classification`, 20000 x 20, `weights=[0.99, 0.01]`, `random_state=42`), split train/validation/test;
- 2) fits `GradientBoostingClassifier` for `learning_rate` in `[1.0, 0.5, 0.1]` with `n_estimators=300`;
- 3) uses `staged_predict` on the validation set to plot F1 vs boosting round for the three learning rates on one chart;
- 4) marks each curve's peak and, for the best setting, quotes test F1 once;
- 5) optional real-data run: `fetch_openml('creditcard')`;
- 6) three-sentence verdict: which eta peaks highest and latest, in P3's melt-the-error language.

[OUTPUT] What to expect

eta = 1.0: fast rise, early peak, then decay (noise)
eta = 0.1: slow steady melt, highest late peak
the staged curve IS your SSE table, drawn live

Your 2-round melt, run for 300 rounds x 3 dials.

Problem Set — Three Things Your Hands Now Know

You computed every number yourself; the code only repeats what you already understand

1

Gini: a question's value is the purity it buys

Root 0.48 $\rightarrow$ 0.16 for the income split (gain 0.32); the card question bought exactly nothing.

2

Bagging: resampled histories, honest disagreement

A6 appeared 0/2/3 times — verdicts D/D/R — and the 2–1 ballot outvoted the fluke.

3

Boosting: half-steps melt the error like clockwork

SSE 1.00 $\rightarrow$ 0.25 $\rightarrow$ 0.0625 at $\eta = 0.5$ — and why the 'perfect' full step is a trap on noisy data.

Hand calculation first — then let Claude Code scale the same idea to real data.